ASP.NET Web API

Tag 1 – REST-Grundlagen und erste API

4-Tage-Kurs | Alexander Fischer, MSc

Organisatorisches

Zeit	
09:00 – 16:30	Kurszeit
Vormittag	15 Minuten Pause
12:00 – 13:00	Mittagspause
Nachmittag	15 Minuten Pause

> Fragen bitte sofort stellen. Eine Frage, die ihr euch stellt, haben meistens drei andere auch.

Vorstellungsrunde

Vorstellungsrunde

- Wie heißt du?

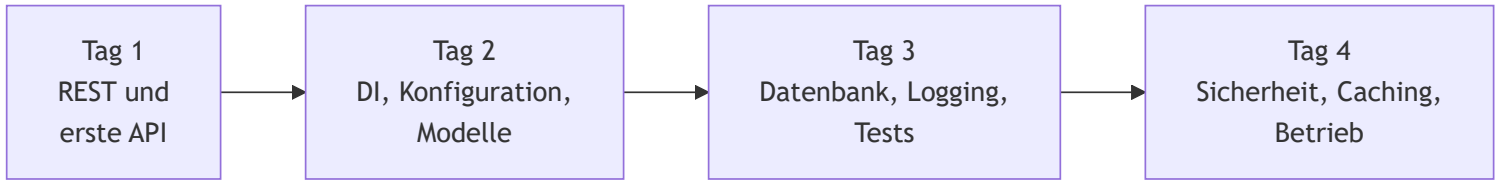
Vorstellungsrunde

- Wie heißt du?
- Was machst du beruflich mit Software?

Vorstellungsrunde

- Wie heißt du?
- Was machst du beruflich mit Software?
- Was willst du nach diesem Kurs können?

Der Kurs in vier Tagen



> Roter Faden: die **Zitate-API**. Jeden Tag kommt nur das Neue dazu.

Ablauf je Thema: Folien → Live-Coding → Übung (10–20 Min.) · nachmittags eine Challenge

Setup-Check

```
dotnet --version
```

- Benötigt: **.NET SDK 8** oder neuer
- Editor: Visual Studio, VS Code mit C# Dev Kit oder Rider
- Alles Weitere kommt per NuGet
- Datenbank: **SQLite**, kein Server nötig

> Fehlermeldung oder Version 6/7? Jetzt melden, nicht erst in der ersten Übung.

Warum REST APIs?

Was ist eine API?

- **A**pplication **P**rogramming **I**nterface
- Programm spricht mit Programm, kein Mensch klickt
- Web API: Kommunikation über **HTTP**
- Jede Sprache und Plattform kann HTTP

> Bild: Restaurant. Die Küche hat die Daten, der Kellner ist die API, die Speisekarte ist die Dokumentation.

REST in einem Satz

REpresentational **S**tate **T**ransfer – ein Architekturstil (Roy Fielding, 2000)

REST in einem Satz

REpresentational **S**tate **T**ransfer – ein Architekturstil (Roy Fielding, 2000)

- **Ressourcen** – alles hat eine eigene Adresse

REST in einem Satz

REpresentational **S**tate **T**ransfer – ein Architekturstil (Roy Fielding, 2000)

- **Ressourcen** – alles hat eine eigene Adresse
- **Einheitliche Schnittstelle** – HTTP-Methoden statt eigener Verben

REST in einem Satz

REpresentational **S**tate **T**ransfer – ein Architekturstil (Roy Fielding, 2000)

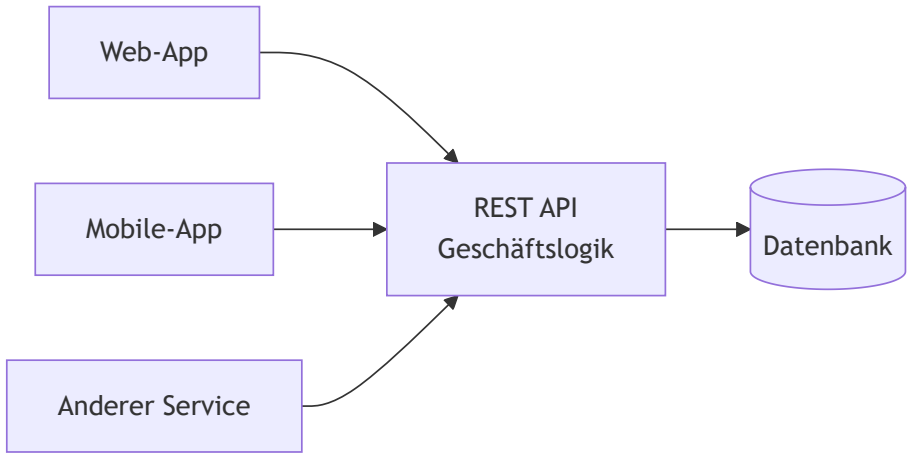
- **Ressourcen** – alles hat eine eigene Adresse
- **Einheitliche Schnittstelle** – HTTP-Methoden statt eigener Verben
- **Zustandslos** – jede Anfrage enthält alles Nötige

REST in einem Satz

REpresentational **S**tate **T**ransfer – ein Architekturstil (Roy Fielding, 2000)

- **Ressourcen** – alles hat eine eigene Adresse
- **Einheitliche Schnittstelle** – HTTP-Methoden statt eigener Verben
- **Zustandslos** – jede Anfrage enthält alles Nötige
- **Repräsentationen** – meist JSON, manchmal XML

REST APIs in Anwendungen



- Logik einmal schreiben, überall nutzen
- Frontend austauschbar, ohne das Backend anzufassen

Ressourcen und URLs

Gut	Schlecht
`GET /api/quotes`	`GET /api/getAllQuotes`
`GET /api/quotes/5`	`GET /api/getQuote?id=5`
`DELETE /api/quotes/5`	`POST /api/deleteQuote/5`
`GET /api/authors/3/quotes`	`GET /api/quotesOfAuthor/3`
`GET /api/quotes?page=2`	`GET /api/quotes/page2`

> Substantive im Plural. Was passieren soll, sagt die Methode, nicht die URL.

HTTP-Methoden

Methode	Zweck	Sicher	Idempotent
`GET`	lesen	ja	ja
`POST`	neu anlegen	nein	nein
`PUT`	komplett ersetzen	nein	ja
`PATCH`	teilweise ändern	nein	nein
`DELETE`	löschen	nein	ja

- **Sicher**: verändert nichts auf dem Server
- **Idempotent**: einmal oder zehnmal – gleiches Ergebnis

Anatomie einer HTTP-Anfrage

Request

```
POST /api/quotes HTTP/1.1
Host: localhost:5001
Accept: application/json
Content-Type: application/json

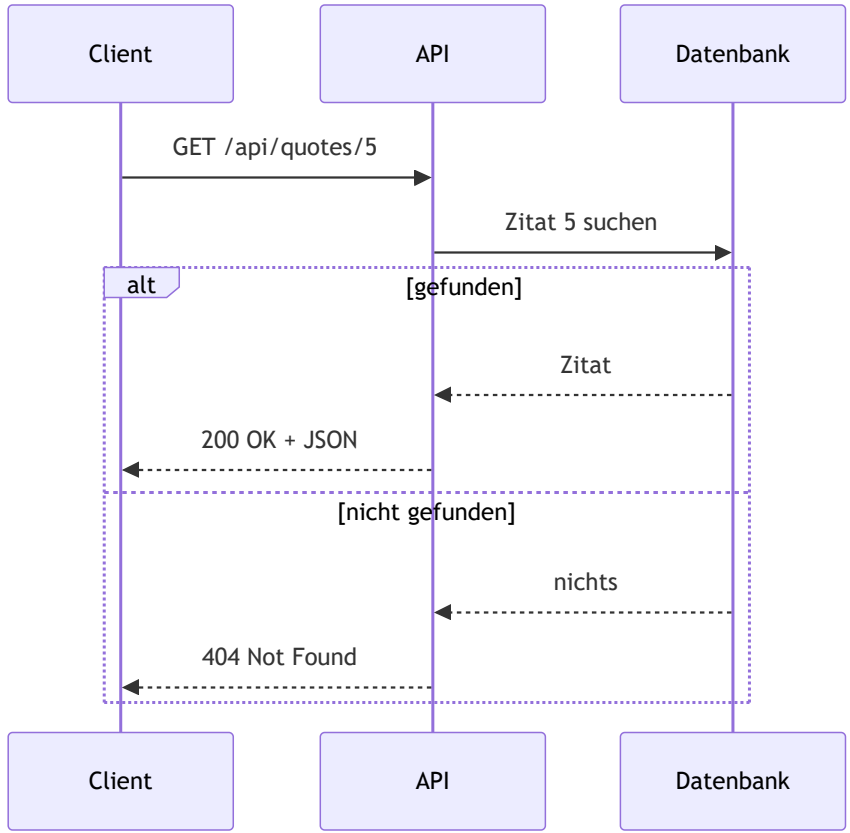
{ "text": "Erst denken, dann coden." }
```

Response

```
HTTP/1.1 201 Created
Location: /api/quotes/3
Content-Type: application/json

{ "id": 3,
  "text": "Erst denken, dann coden." }
```

Request und Response im Ablauf



HTTP Status Codes – die fünf Klassen

Klasse	Bedeutung	Wer muss handeln?
`1xx`	Information	–
`2xx`	Erfolg	niemand
`3xx`	Umleitung	Client folgt
`4xx`	**Fehler des Clients**	Aufrufer ändert die Anfrage
`5xx`	**Fehler des Servers**	Betreiber repariert

Die wichtigsten Status Codes

Code	Bedeutung	Code	Bedeutung
`200`	OK	`403`	Forbidden
`201`	Created	`404`	Not Found
`204`	No Content	`409`	Conflict
`400`	Bad Request	`429`	Too Many Requests
`401`	Unauthorized	`500`	Internal Server Error

Welcher Code passt?

Situation	Code
Zitat mit Id 99 gibt es nicht	
Zitat erfolgreich angelegt	
Im JSON fehlt der Text	
Datenbank nicht erreichbar	
Zitat gelöscht	

Welcher Code passt?

Situation	Code
Zitat mit Id 99 gibt es nicht	`404`
Zitat erfolgreich angelegt	
Im JSON fehlt der Text	
Datenbank nicht erreichbar	
Zitat gelöscht	

Welcher Code passt?

Situation	Code
Zitat mit Id 99 gibt es nicht	`404`
Zitat erfolgreich angelegt	`201`
Im JSON fehlt der Text	
Datenbank nicht erreichbar	
Zitat gelöscht	

Welcher Code passt?

Situation	Code
Zitat mit Id 99 gibt es nicht	`404`
Zitat erfolgreich angelegt	`201`
Im JSON fehlt der Text	`400`
Datenbank nicht erreichbar	
Zitat gelöscht	

Welcher Code passt?

Situation	Code
Zitat mit Id 99 gibt es nicht	`404`
Zitat erfolgreich angelegt	`201`
Im JSON fehlt der Text	`400`
Datenbank nicht erreichbar	`500`
Zitat gelöscht	

Welcher Code passt?

Situation	Code
Zitat mit Id 99 gibt es nicht	`404`
Zitat erfolgreich angelegt	`201`
Im JSON fehlt der Text	`400`
Datenbank nicht erreichbar	`500`
Zitat gelöscht	`204`

Entwicklungsumgebung und Projekt

Entwicklungsumgebungen

Werkzeug	Plattform	Besonderheit
Visual Studio	Windows	große IDE, Community Edition kostenlos
VS Code + C# Dev Kit	alle	leichtgewichtig
Rider	alle	JetBrains, nicht-kommerziell kostenlos

> Alles im Kurs geht mit der **dotnet CLI** im Terminal – für alle gleich.

Ein Projekt anlegen

```
mkdir tag1
cd tag1
dotnet new sln -n Tag1
dotnet new webapi --use-controllers -n KursApi
dotnet sln add KursApi
cd KursApi
dotnet run
```

- `webapi` – die Vorlage
- `--use-controllers` – Controller-Klassen statt Minimal APIs
- `-n KursApi` – Projektname
- Port steht in der Konsole und in `Properties/launchSettings.json`

Typische Projektstruktur

```
KursApi/
├── Controllers/
│   └── WeatherForecastController.cs
├── Properties/
│   └── launchSettings.json     Port, Umgebung
├── appsettings.json            Konfiguration
├── appsettings.Development.json
├── KursApi.csproj              Framework, Pakete
├── Program.cs                  Einstiegspunkt
└── bin/ obj/                   Build – nicht in Git
```

Später dazu: `Models/`, `Services/`, `Data/`

Program.cs – zwei Phasen

```
var builder = WebApplication.CreateBuilder(args);

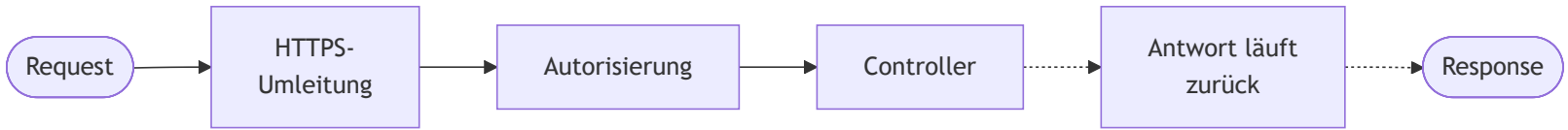
// Phase 1: Dienste registrieren
builder.Services.AddControllers();

var app = builder.Build();

// Phase 2: Pipeline konfigurieren
app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();

app.Run();
```

Die Middleware-Pipeline



- Jede Middleware tut etwas und reicht weiter – oder antwortet selbst
- **Die Reihenfolge zählt**

Packagemanagement mit NuGet

Was ist NuGet?

- Paketmanager für .NET (wie npm oder Maven)
- Über 400.000 Pakete auf **nuget.org**
- Eintrag in der `.csproj` als `PackageReference`
- Pakete werden beim Build geladen – nur die Referenz kommt in Git

```
<ItemGroup>
  <PackageReference Include="Humanizer" Version="2.14.1" />
</ItemGroup>
```

NuGet-Befehle

```
dotnet add package Humanizer            # hinzufügen
dotnet add package Humanizer --version 2.14.1
dotnet list package                     # anzeigen
dotnet remove package Humanizer         # entfernen
```

> Die Paketversion muss zur .NET-Version passen. Unter .NET 8 z. B. `--version 8.*`

Übung 1 – Projekt anlegen und NuGet

15 Minuten

1. Ordner `tag1` mit Solution `Tag1` anlegen, darin Projekt `KursApi` mit `--use-controllers`
2. Starten und `/weatherforecast` im Browser aufrufen
3. Ein NuGet-Paket hinzufügen
4. In der `.csproj` nachsehen
5. Das Paket wieder entfernen

Bonus: In `launchSettings.json` den Port ändern

Lösung Übung 1

```
mkdir tag1
cd tag1
dotnet new sln -n Tag1
dotnet new webapi --use-controllers -n KursApi
dotnet sln add KursApi
cd KursApi
dotnet run
# Browser: https://localhost:<port>/weatherforecast

dotnet add package Humanizer
# KursApi.csproj: <PackageReference Include="Humanizer" ... />
dotnet remove package Humanizer
```

Zertifikatwarnung? `dotnet dev-certs https --trust`

API-Controller

Controller und Aktionen

```
[ApiController]
[Route("api/[controller]")]
public class HelloController : ControllerBase
{
    [HttpGet]
    public string Get() ⇒ "Hallo Welt";
}
```

- **Controller**: Klasse, die Endpoints einer Ressource bündelt
- **Aktion**: öffentliche Methode, die einen Endpoint bedient

ControllerBase und [ApiController]

Zutat	Wirkung
`: ControllerBase`	Hilfsmethoden `Ok()`, `NotFound()`, `BadRequest()` …
`[ApiController]`	automatische 400, Bindungsregeln, ProblemDetails
`[Route("…")]`	Basis-URL des Controllers
Name endet auf `Controller`	wird automatisch gefunden

`Controller` (ohne Base) bringt HTML-Views mit – für APIs nicht nötig.

Attribut-Routing

```
[Route("api/[controller]")]
public class HelloController
    : ControllerBase
{
    [HttpGet]
    public string Get() ⇒ "...";

    [HttpGet("{name}")]
    public string Get(string name)
        ⇒ "...";
}
```

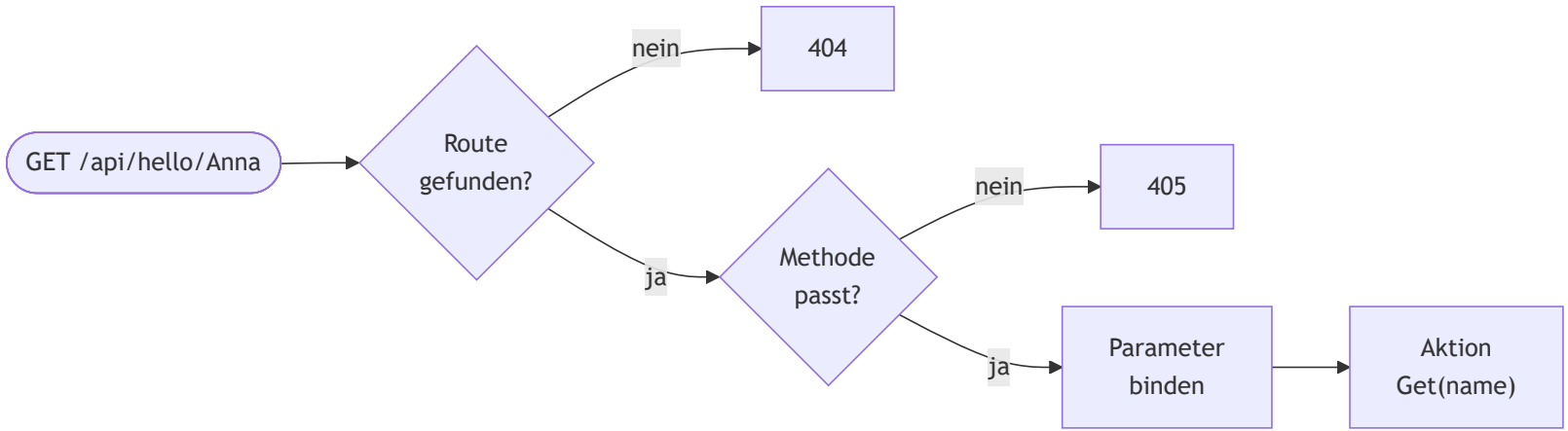
Aktion	Route
`Get()`	`GET /api/hello`
`Get(name)`	`GET /api/hello/{name}`

`[controller]` = Klassenname ohne „Controller“. Routen sind nicht case-sensitive.

Platzhalter und Constraints

Route	passt auf	passt nicht
`{id}`	`/5`, `/abc`	–
`{id:int}`	`/5`	`/abc` → 404
`{id:guid}`	`/3f2a…`	`/5`
`{id:min(1)}`	`/1`, `/7`	`/0`
`{name:length(3,20)}`	`/Anna`	`/Al`
`{id?}`	`/`, `/5`	–

Von der URL zur Aktion



Übung 2 – Erster eigener Controller

10 Minuten

Erstelle einen `HelloController` mit:

- `GET /api/hello` → „Hallo Welt“
- `GET /api/hello/{name}` → „Hallo “

Bonus: `GET /api/hello/{name}/{times:int}` wiederholt die Begrüßung

> Tipp: `dotnet watch` baut bei jeder Änderung automatisch neu.

Lösung Übung 2

```
[ApiController]
[Route("api/[controller]")]
public class HelloController : ControllerBase
{
    [HttpGet]
    public string Get() ⇒ "Hallo Welt";

    [HttpGet("{name}")]
    public string Get(string name) ⇒ $"Hallo {name}";
}
```

Parameterbindung

Woher kommen die Daten?

Quelle	Attribut	Beispiel	Standard bei `[ApiController]`
Route	`[FromRoute]`	`/quotes/5`	Name steht in der Route
Query	`[FromQuery]`	`?page=2`	einfache Typen
Body	`[FromBody]`	JSON	komplexe Typen
Header	`[FromHeader]`	`X-Api-Version`	nie automatisch

> Höchstens **ein** Body-Parameter pro Aktion.

Parameterbindung im Code

```
[HttpGet("{id:int}")]
public ActionResult<Quote> Get(
    int id,                                  // Route
    [FromQuery] bool includeAuthor,          // ?includeAuthor=true
    [FromHeader(Name = "X-Api-Version")]
    string? version)                         // Header
{ ... }

[HttpPost]
public ActionResult<Quote> Create(Quote quote)   // Body
{ ... }
```

Rückgabetypen

Rückgabetyp	Status steuerbar	Swagger kennt den Typ
`string` , `Quote`	nein, immer 200	ja
`IActionResult`	ja	nein
`ActionResult<T>`	ja	ja

```
public ActionResult<Quote> GetById(int id)
{
    if (id ⩽ 0) return NotFound();    // 404
    return new Quote { Id = id };     // 200 + JSON
}
```

Status Codes zurückgeben

Hilfsmethode	Status
`Ok(wert)`	200
`CreatedAtAction(name, routeValues, wert)`	201 + Location
`NoContent()`	204
`BadRequest("Meldung")`	400
`NotFound()`	404
`Conflict("Meldung")`	409
`StatusCode(418)`	beliebig

Übung 3 – Parameterbindung und Status Codes

15 Minuten

`CalcController` mit:

- `GET /api/calc/add?a=3&b=4`
- `GET /api/calc/divide?a=10&b=2`
- Division durch 0 → **400 Bad Request**

Bonus: `POST /api/calc/sum` mit Zahlen-Array im Body, z. B. `[1, 2, 3]`

Lösung Übung 3

```
[HttpGet("add")]
public int Add([FromQuery] int a,
               [FromQuery] int b)
    => a + b;

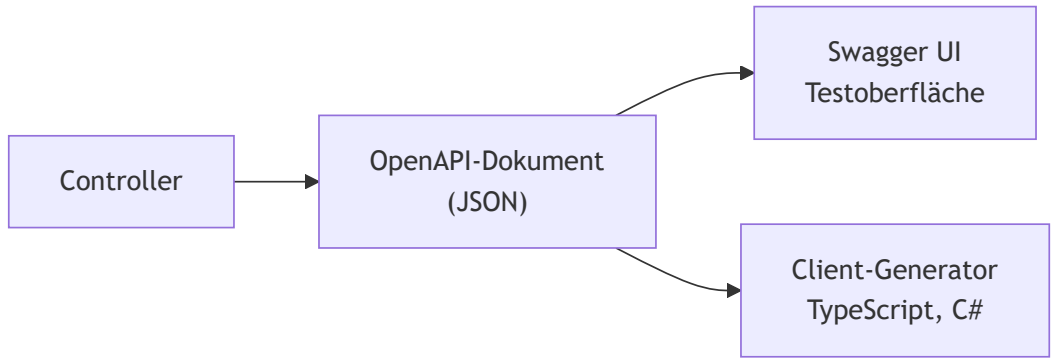
[HttpGet("divide")]
public ActionResult<double> Divide(
    int a, int b)
{
    if (b == 0)
        return BadRequest(
          "Division durch 0 ist nicht erlaubt.");
    return (double)a / b;
}
```

```
[HttpPost("sum")]
public int Sum(
    [FromBody] int[] numbers)
    => numbers.Sum();
```

Ohne `(double)` rechnet C# ganzzahlig: 7 / 2 = 3

Dokumentation mit OpenAPI/Swagger

Was ist OpenAPI?



- **OpenAPI**: Standard, der eine REST-API maschinenlesbar beschreibt
- **Swagger UI**: klickbare Oberfläche zum Testen

Swagger einrichten (1/2)

.NET 8: Swashbuckle ist schon dabei · ab .NET 9: nur `/openapi/v1.json`, Paket nachinstallieren

```
dotnet add package Swashbuckle.AspNetCore
```

XML-Kommentare aktivieren – `KursApi.csproj`, in der `PropertyGroup`:

```
<GenerateDocumentationFile>true</GenerateDocumentationFile>
<NoWarn>$(NoWarn);1591</NoWarn>
```

Swagger einrichten (2/2)

```
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen(o ⇒
{
    var xml = Path.Combine(AppContext.BaseDirectory, "KursApi.xml");
    o.IncludeXmlComments(xml);
});
// ...
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();   // erreichbar unter /swagger
}
```

XML-Kommentare

```
/// <summary>Begrüßt eine Person mit Namen.</summary>
/// <param name="name">Name der Person</param>
/// <returns>Die Begrüßung</returns>
/// <response code="200">Begrüßung erfolgreich</response>
[HttpGet("{name}")]
public string Get(string name) ⇒ $"Hallo {name}";
```

> Drei Schrägstriche über einer Methode – die IDE erzeugt das Gerüst.

curl in einer Minute

Option	Bedeutung
`-X POST`	HTTP-Methode
`-H "…"`	Header setzen
`-d '…'`	Body senden
`-i`	Status und Header anzeigen
`-k`	Entwicklungszertifikat ignorieren

```
curl -k -i https://localhost:<port>/api/hello/Anna
```

Windows PowerShell: `curl.exe` statt `curl`

Übung 4 – Swagger UI und XML-Kommentare

15 Minuten

1. Swagger UI einbinden (je nach .NET-Version)
2. Alle bisherigen Endpoints über Swagger testen
3. Mindestens zwei Aktionen mit XML-Kommentaren versehen
4. Einen Endpoint zusätzlich mit `curl` aufrufen

Lösung Übung 4

- Paket installiert (ab .NET 9) oder schon vorhanden (.NET 8)
- `GenerateDocumentationFile` und `NoWarn` in der `.csproj`
- `AddSwaggerGen` mit `IncludeXmlComments`
- `UseSwagger()` und `UseSwaggerUI()` in Development
- `/swagger` zeigt alle Controller mit Kommentaren

> Seite leer? Netzwerktab prüfen: `swagger.json` liefert die Fehlermeldung.

Challenge 1 – Zitate-API

30 Minuten

Baue einen `QuotesController` mit einer **statischen Liste** von Zitaten.

Verwendet: Controller, Routen, Parameterbindung, Status Codes, Swagger

> Warum `static`? Für jede Anfrage wird ein neuer Controller erzeugt.

Challenge 1 – Anforderungen

Endpoint	Antwort
`GET /api/quotes`	alle Zitate, 200
`GET /api/quotes/{id}`	ein Zitat, 200 oder 404
`POST /api/quotes`	201 Created mit Location-Header
Bonus: `DELETE /api/quotes/{id}`	204 oder 404

Schnell fertig? `PUT /api/quotes/{id}` oder `GET /api/quotes/random`

Lösung Challenge 1 (1/3)

```
public class Quote
{
    public int Id { get; set; }
    public string Text { get; set; } = "";
}

[ApiController]
[Route("api/[controller]")]
public class QuotesController : ControllerBase
{
    private static readonly List<Quote> _quotes = new()
    {
        new Quote { Id = 1, Text = "Erst denken, dann coden." },
        new Quote { Id = 2, Text = "Auf meinem Rechner läuft es." }
    };
```

Lösung Challenge 1 (2/3)

```
    [HttpGet]
    public ActionResult<IEnumerable<Quote>> GetAll() ⇒ Ok(_quotes);

    [HttpGet("{id:int}")]
    public ActionResult<Quote> GetById(int id)
    {
        var quote = _quotes.FirstOrDefault(q ⇒ q.Id == id);
        if (quote is null) return NotFound();
        return quote;
    }
```

Lösung Challenge 1 (3/3)

```
[HttpPost]
public ActionResult<Quote> Create(
    Quote quote)
{
    quote.Id = _quotes.Count == 0
        ? 1 : _quotes.Max(q => q.Id) + 1;
    _quotes.Add(quote);
    return CreatedAtAction(
        nameof(GetById),
        new { id = quote.Id }, quote);
}
```

```
[HttpDelete("{id:int}")]
public IActionResult Delete(int id)
{
    var quote = _quotes
        .FirstOrDefault(q => q.Id == id);
    if (quote is null) return NotFound();
    _quotes.Remove(quote);
    return NoContent();
}
```

Rückblick Tag 1

Rückblick Tag 1

- REST: Ressourcen, Methoden, Status Codes

Rückblick Tag 1

- REST: Ressourcen, Methoden, Status Codes
- Projekt mit der dotnet CLI, Struktur, Pipeline in `Program.cs`

Rückblick Tag 1

- REST: Ressourcen, Methoden, Status Codes
- Projekt mit der dotnet CLI, Struktur, Pipeline in `Program.cs`
- NuGet

Rückblick Tag 1

- REST: Ressourcen, Methoden, Status Codes
- Projekt mit der dotnet CLI, Struktur, Pipeline in `Program.cs`
- NuGet
- Controller, Routen, Parameterbindung, Rückgabetypen

Rückblick Tag 1

- REST: Ressourcen, Methoden, Status Codes
- Projekt mit der dotnet CLI, Struktur, Pipeline in `Program.cs`
- NuGet
- Controller, Routen, Parameterbindung, Rückgabetypen
- Swagger und curl

Rückblick Tag 1

- REST: Ressourcen, Methoden, Status Codes
- Projekt mit der dotnet CLI, Struktur, Pipeline in `Program.cs`
- NuGet
- Controller, Routen, Parameterbindung, Rückgabetypen
- Swagger und curl
- Eine laufende Zitate-API

Ausblick Tag 2

Services und Dependency Injection · appsettings · Validierung · Versionierung

Bitte das Projekt von heute mitbringen – morgen kopieren wir es in `tag2` und arbeiten dort weiter.



Mühlehner & Tavolato GmbH

SCHULUNG · BERATUNG · ENTWICKLUNG

AF

Alexander Fischer

af@mit.at

Kurse

offen und als Firmenschulung

Individualschulungen

zugeschnitten auf euer Team

Consulting

Beratung für eure Projekte

Auftragsentwicklung

wir bauen eure Software